School of Engineering, Arts, Science and Technology

INTRODUCTION TO RELATIONAL DATABASES ASSESSMENT 1: DATABASE DESIGN

STUDENT NUMBER

S183038

Word Count: 1039

Table of Contents

Introduction

This report documents the design of a relational database management system (RDMS) for a new online video game marketplace to compete with "Steam". Users will be able to play and communicate with friends, purchase games, and write reviews for their games.

The design goes through the normalization process with the aim of minimizing data redundancy and maintaining consistency across the system.

Requirements

The system should be designed to be scalable, secure and can add additional features later. It should meet the following data requirements as minimum:

- User Account Management: Registration, login, profile management, and secure password storage.
- Game Management: Support for digital game purchases, system requirements, pricing, and genre classification.
- Social Features: Friend connections, chat rooms, and private messaging.
- User Reviews: Ratings and written reviews on owned games.
- Moderation Tools: Banning users from games, logging reasons and expiry times.

Database Design and Normalisation

Conceptual ERD

A Conceptual ERD was developed to map out all the entities and their relationships (*Figure 1*). Entities include Account, Game, Chat, Message, Review, Game Ban, Owned Game, Platform, and System Requirement.

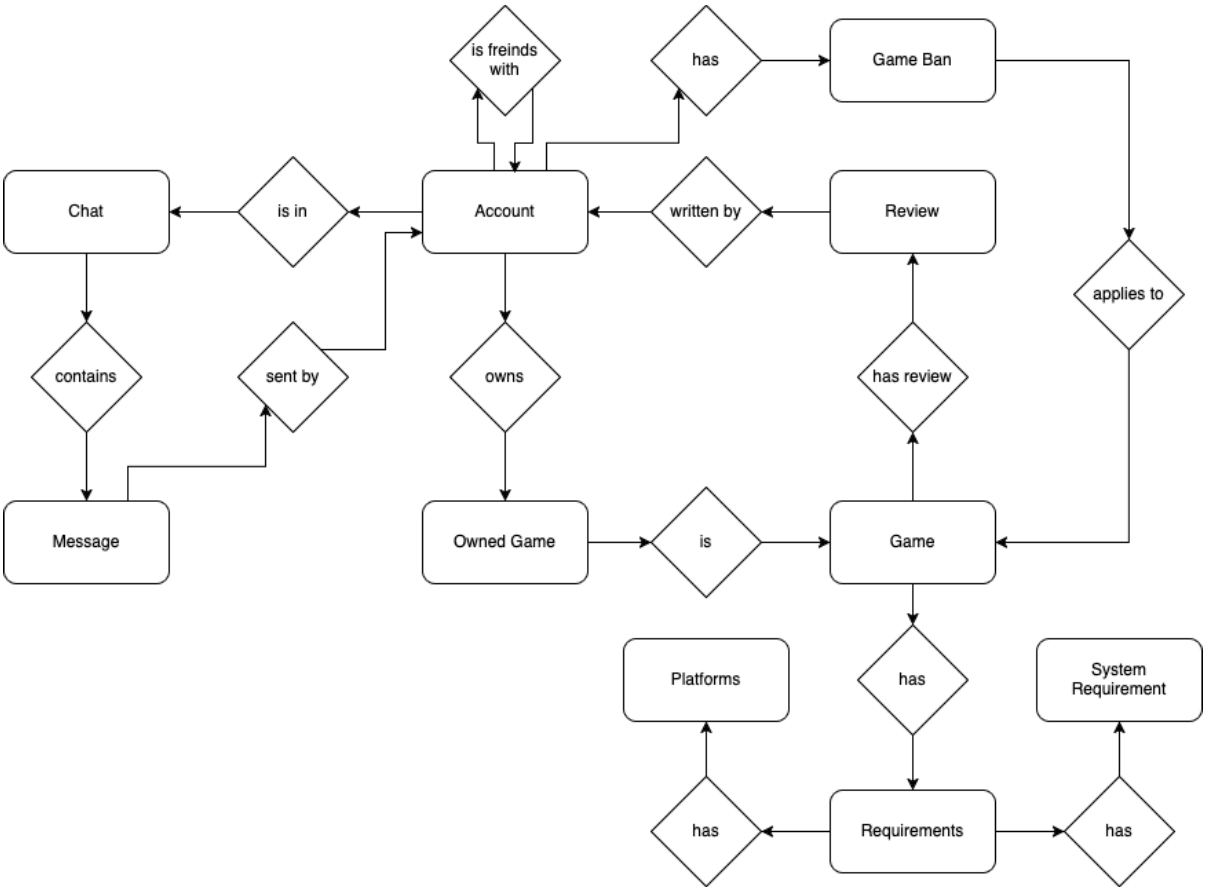

Figure 1: Conceptual ERD

Unnormalized Form (UNF / 0NF)

The first model was created in Unnormalized Form (0NF), all data was held in a single table (account) with attributes such as game_bans, has_games, friends, chats, messages, and reviews stored as JSON fields.

This presented the following possible data anomalies:

- Insertion anomalies: A game cannot be added without an account.
- Update anomalies: Changing a game name requires multiple updates.
- Deletion anomalies: Deleting an account could also delete all evidence of a game, if it was the only account to have that game.

The simple ERD is shown in *figure 2*.

account	
email	VARCHAR(100)
username	VARCHAR(75)
password	VARCHAR(100)
profile_pic_url	VARCHAR(200)
account_banned	BOOLEAN
game_bans	JSON
has_games	JSON
freinds	JSON
chats	JSON
messages	JSON

Figure 2: 0NF ERD

First Normal Form (1NF)

To transition my data into first normal form, all attributes needed to be made atomic and repeating groups had to be eliminated. (Elmasri and Navathe 2016, p. 507). Key design changes included:

- Extracting the repeating JSON fields into separate tables, such as friends, reviews, messages and owned_games.
- Ensuring that each table had a primary key:
    - account.email and game.name were used as natural primary keys.
    - "Surrogate keys" (Elmasri and Navathe 2016, p. 204) were used in other tables where no natural primary key was present.
- Many-to-many relationships were represented with the use of 'junction' tables, also known as associative entities (Hoffer, Prescott & McFadden 2004, p. 143). these tables include:
    - 'owned_games'
    - 'game_genres'
    - 'chat_users'

The design shown in *figure 3* satisfies the requirements for 1NF.

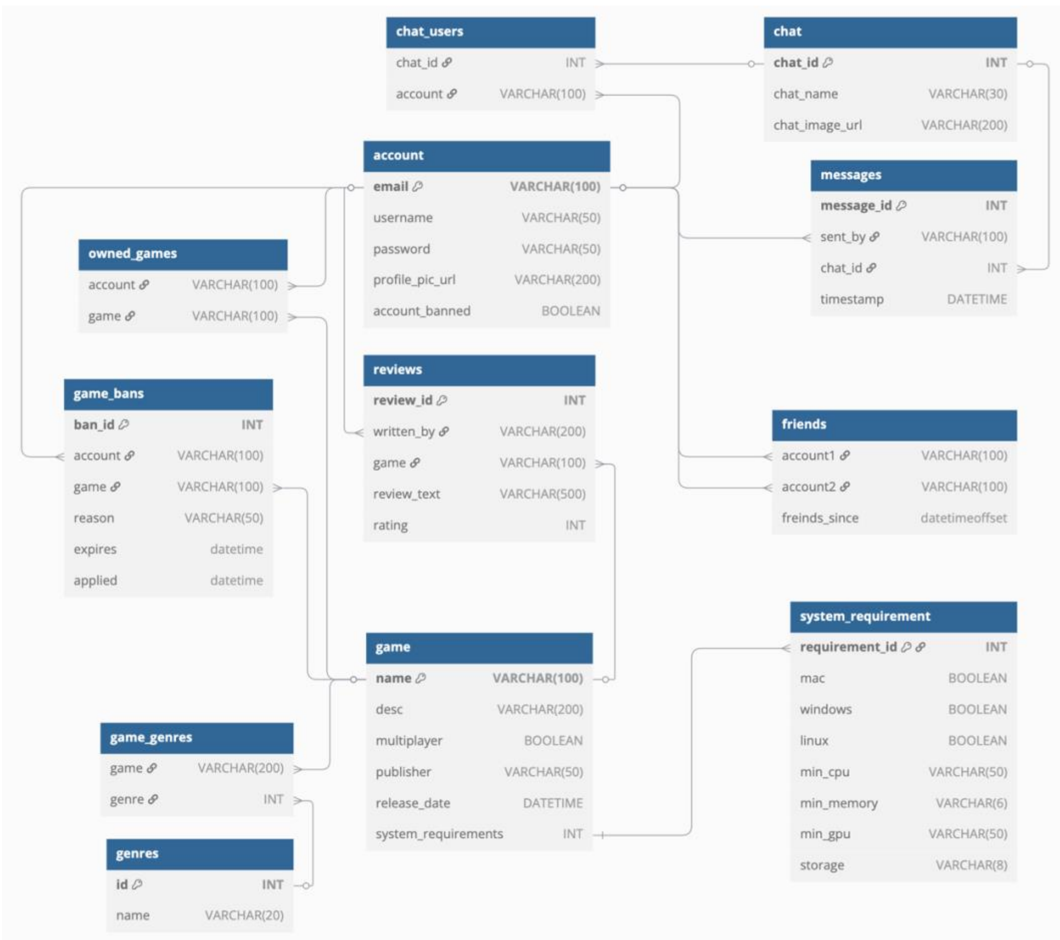

Figure 3: 1NF -> 2NF -> 3NF Logical ERD Diagram

Second Normal Form (2NF)

2NF requires that there are no partial dependencies on a composite primary key (Elmasri and Navathe 2016, p. 512).

- In ‘friends’ the attribute ‘friends_since’ relies on the whole composite key of ‘account1’ + ‘account2’, meeting the requirement.
- In the other junction tables (‘chat_users’, ‘owned_games’ and ‘game_genres’), the table only consists of the composite primary key with no dependencies. Also meeting the requirement.

Since no tables had any partial dependencies, my design already met 2NF.

Third Normal Form (3NF)

3NF requires that there are no transitive dependencies (non-key attributes that depend on other non-key attributes) (Elmasri and Navathe 2016, p. 516). Fortunately, no such dependencies existed.

For example, the ‘game’ table, all non-key attributes directly depend on the whole primary key (‘name’). The same applying for all other tables, meaning it was compliant with 3NF.

Boyce-Codd Normal Form (BCNF) and Final Design.

Finally, the design was made compliant with BCNF**.** Requiring that every determinant is a candidate key, ensuring that there are no anomalies caused by functional dependencies where non-candidate keys determine other attributes (Elmasri and Navathe 2016, p. 518). Key changes made here include:

- Introduction of surrogate keys for ‘account’ and ‘game’ as auto incrementing integers, this ensures data integrity if the user updates their email, or the name of a game is changed. This satisfies the requirements for BCNF since the only determinant is ‘game_id’ and no attribute is determined by anything that isn’t a candidate key.
- All references to account.email and game.name were updated to reflect their new surrogate primary keys.

I also separated publisher into a separate table, allowing for additional functionality. This allows authenticated users to manage games and bans.

Figure 4 shows the final ERD diagram for my design.

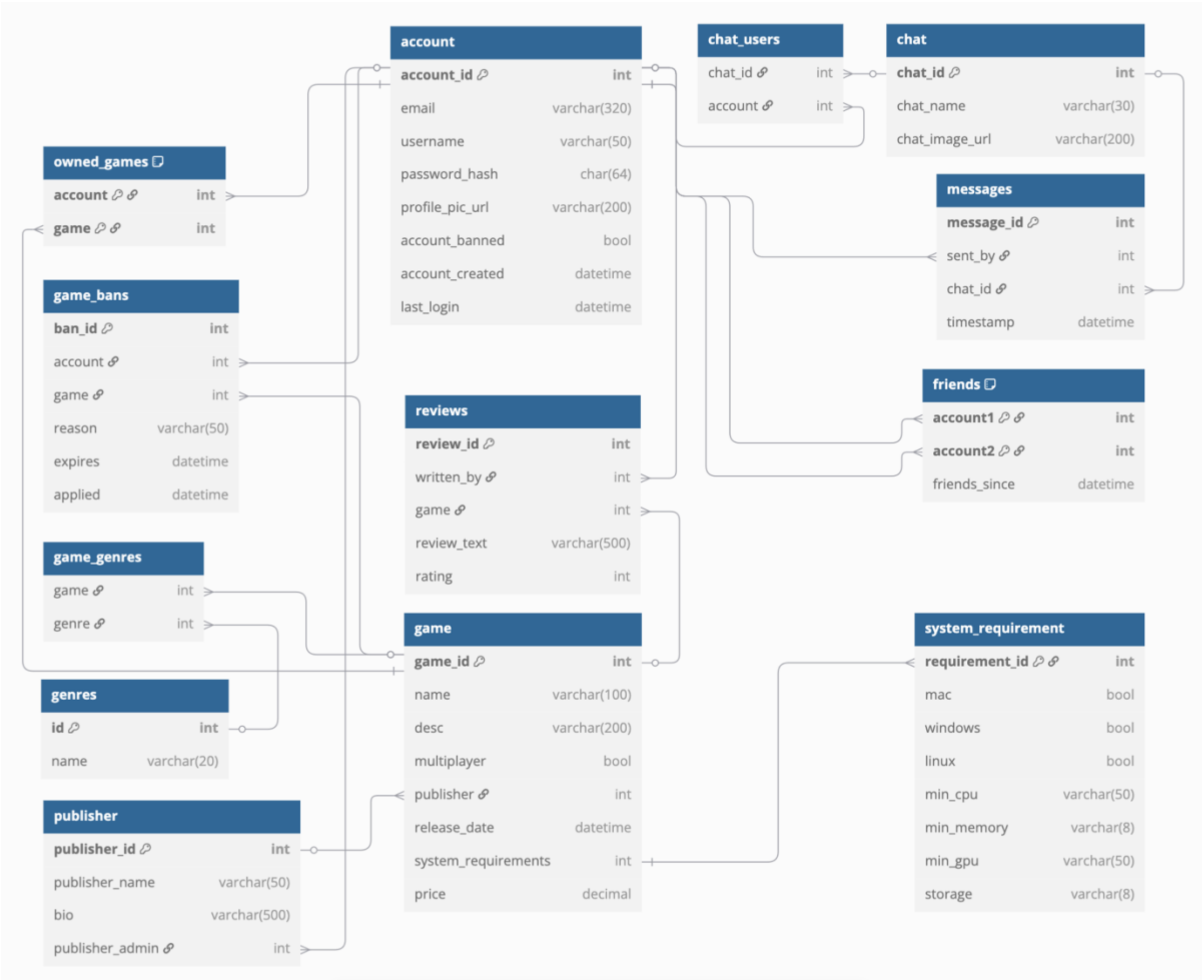

Figure 4: Final BCNF Logical ERD Diagram

Data Types, Constraints and Security Considerations

The database uses data types and constraints appropriate to the fields purpose and expected data. This is a list of examples using fields from various tables:

Field (table name)	Data Type	Constraints
account_id (account)	INT (PK)	AUTO_INCREMENT
email (account)	VARCHAR(320)	UNIQUE, NOT NULL
username (account)	VARCHAR(50)	NOT NULL
password_hash (account)	CHAR(64)	NOT NULL
price (game)	DECIMAL	NOT NULL
friends_since (friends)	DATETIME	NOT NULL
publisher (game table)	INT (FK)	FOREIGN KEY OR NULL

Constraints:

- Primary Keys (PK): Uniquely identify each record.

- Foreign Keys (FK): Enforce referential integrity between tables.
- NOT NULL: Ensure mandatory data fields are populated.
- UNIQUE: Prevent duplicate entries, e.g., in email.
- Composite Keys: Used in junction tables like owned_games.
- Passwords are securely hashed using SHA-256, which produces a 64-character hash when represented in hexadecimal format; therefore, a CHAR(64) field is used to store the hash (Yao, Dodis & Wichs, 2013).
- Email is stored with a limit of 320 characters, following RFC standards (Resnick 2008).

Future Enhancements and Potential Issues

The database is designed in a way where it should be easy to add additional functionality later if required, some examples of this could be:

- Game achievements earned in game
- Downloadable content (DLC)

If an account is deleted it may delete records that are required for legal purposes such as where a game was purchased. This system could be improved on by adding some form of backup for this kind of sensitive data.

Potential User Queries

Below are some potential queries that might be made to this database in simple terms:

Query	**Expected Result**
Create account	A new entry is made into the account table
Login	A provided email and password are compared against a matching account entry
View store	A list of all available games for purchase and their prices
View Library	A list of all games that the account owns
Show friends	A list of all friends that an account has
Send Message	A message entry added with a reference to a chat using a foreign key

Conclusion

The final design meets the requirements outlined in the introduction. It is normalised to BCNF, addresses key security concerns such as password storage, and considers potential future developments and improvements.

Bibliography

Elmasri, R. & Navathe, S., 2016. *Fundamentals of Database Systems*, Global Edition, Pearson Education, Limited, Harlow. Available from: ProQuest Ebook Central. [Accessed 13 March 2025].

Hoffer, J.A., Prescott, M.B. & McFadden, F.R., 2004. *Modern Database Management*, 7th edn, Prentice Hall, New Jersey.

Resnick, P., 2008. *RFC 5322: Internet message format*. IETF. Available at: https://datatracker.ietf.org/doc/html/rfc5322 [Accessed 13 March 2025].

Yao, X., Dodis, Y. & Wichs, D., 2013. Security analysis of salt||password hashes. In: *IEEE 2013 International Conference on Information Theory (ITW)*. IEEE. Available at: https://ieeexplore.ieee.org/document/6516321 [Accessed 13 March 2025].